[TCW] - 3rd Party Research - TCW-70 - 2026-08-04

Transactional email delivery integration research for TCW-70

Primary recommendation: Amazon Simple Email Service (Amazon SES)

Integration estimate: 10-15 engineering days | Modeled 10,000-MAU cost: $5.60/month*

**Assumes 3.5 transactional emails per active user per month (35,000 emails/month), excluding attachment data, premium plans, and support.*

Project Overview

RelayForge is a multi-tenant B2B customer-operations platform for returns, warranty claims, refunds, and shipment exceptions. TCW-70 requires provider-neutral transactional customer email without changing the PostgreSQL outbox, NestJS/BullMQ worker boundary, or synchronous case-creation path.

- Scope: account verification, return authorization, shipping-label-ready, refund status, warranty-case, and human-agent transactional messages; bulk and marketing email are excluded.
- Operating model: approximately 180 merchants in Canada and the United States, with a later European Union deployment path.
- Capacity: 35,000 messages per business day, 25 submissions/second normally, and 90 submissions/second for ten-minute peaks.
- Governance: vendor types remain behind TransactionalMessagePort; retries, signatures, idempotency, monitoring, privacy controls, and failure mappings are provider-neutral.
- Jira project description: Not specified in source.

Supporting source: RelayForge Transactional Email Integration — Research Source Pack

Feature Requirements

Ticket-derived functional requirements

- Support tenant-scoped configuration and verified sender identities for about 180 merchants. Source: TCW-70 description.
- Send six named transactional notification families in HTML and plain text from versioned en and fr-CA templates with tenant branding. Source: TCW-70 description and acceptance criteria.

- Support metadata plus attachments or time-limited links for return labels and warranty documents. Source: TCW-70 description and acceptance criteria.
- Normalize accepted, delivered, deferred, bounced, complained, and rejected events into searchable case audit history. Source: TCW-70 description and acceptance criteria.
- Allow per-tenant disablement and a safe vendor test environment. Source: TCW-70 description.

Ticket-derived non-functional requirements

- Handle 35,000 messages/day, 25 submissions/second steady state, and 90 submissions/second for ten-minute peaks. Source: TCW-70 description.
- Run only in the NestJS 11 BullMQ worker on Node.js 22 and Linux/AMD64 ECS Fargate, with outbound HTTPS, no persistent disk, and five-second deadlines. Source: TCW-70 description.
- Use bounded asynchronous retries, idempotent callbacks, tenant scoping, and tolerance for duplicate or out-of-order events. Source: TCW-70 description and acceptance criteria.
- Preserve the PostgreSQL outbox as system of record and prohibit browser or synchronous API provider calls. Source: TCW-70 description.
- Document assumptions, gaps, implementation risks, and follow-up technical spikes. Source: TCW-70 acceptance criteria.

Ticket-derived security requirements

- Authenticate callbacks and define replay protection, credential separation, failure mapping, idempotency, timeout, and retry ownership. Source: TCW-70 description and acceptance criteria.
- Keep vendor-specific fields behind the communications-domain port. Source: TCW-70 acceptance criteria.
- Cover provider failures, regional processing, privacy, retention, and audit controls. Source: TCW-70 acceptance criteria.

Ticket-derived compliance requirements

- Support privacy and data residency for Canada and the United States and a future EU deployment. Source: TCW-70 description and acceptance criteria.
- Preserve auditable provider-neutral status history and define retention behavior. Source: TCW-70 acceptance criteria.

Linked source-pack findings

- Implementation restrictions: one isolated adapter; strict TypeScript; local fakes and contract tests; deterministic lockfile updates; Docker Compose and sandbox only in non-production. Source: Research Source Pack.

- UI dependency: no frontend or synchronous API provider call; provider-neutral status must update within two minutes. Source: Research Source Pack.
- Security: TLS 1.2+, encryption at rest, least-privilege credentials, quarterly rotation, dual-key overlap, raw-body signature validation, five-minute replay window, and no sensitive logs. Source: Research Source Pack.
- Compliance: processing-location disclosure, Canada/US privacy terms, erasure/suppression, 30-day provider metadata target, and 13-month RelayForge audit retention. Source: Research Source Pack.
- Performance: API queue p95 below 150 ms, handoff p95 below 60 seconds and p99 below five minutes, six-hour transient retry window. Source: Research Source Pack.
- Integration assumptions: at-least-once delivery, dedupe by provider event ID/source, event-time ordering, 512 KiB webhook body, and quarantine unknown events. Source: Research Source Pack.

The source pack was inside the confirmed Drive folder and fully readable. TCW-70 has no direct attachments and no remote links; no inaccessible or out-of-scope document was encountered.

Technology Stack

Dimension	Detected value
Project overview	Multi-tenant B2B operations platform; modular monolith plus independently scaled workers
Frontend	Next.js 15.3.3, React 19.1.0, TanStack Query 5.80
Backend	NestJS 11.1.3 on Express; OpenAPI 3; Prisma 6.9
Database	PostgreSQL 16 on RDS Multi-AZ
Storage	Amazon S3 with KMS and signed URLs; MinIO locally
Queues/cache	Redis 7 / ElastiCache; BullMQ 5.53.2
Runtime	Node.js 22.15.1; TypeScript 5.7.3; Linux/AMD64
Authentication	OIDC/OAuth 2.1 Authorization Code + PKCE; access tokens; tenant-scoped RBAC
Deployment	Docker to ECR; AWS ECS Fargate; Terraform
Cloud platform	AWS ALB, CloudFront, WAF, RDS, ElastiCache, S3, Secrets Manager, KMS, CloudWatch
Third-party services	Stripe, Avalara, Shippo; transactional email not implemented

Dimension	Detected value
Package manager	pnpm 10.12.1 via Corepack
Build tool	Turborepo 2.5.4, Nest CLI, Next.js, GitHub Actions
Coding style	Strict TypeScript, ESLint 9, Prettier 3.5, conventional commits, Vitest, provider-neutral ports

Repository configuration at commit 8b4c93b20b2949ea95414e81b95fe0f1c10f0fe1 overrides narrative documentation when inconsistent.

Evaluation Methodology

Repository snapshot: default branch main at commit 8b4c93b20b2949ea95414e81b95fe0f1c10f0fe1. Every repository read in this run used that SHA.

Vendor-research as-of date: Aug 4, 2026

Adoption signals were limited to official Node SDK/client GitHub stars, relevant npm weekly downloads for 2026-07-28 through 2026-08-03, and one third-party category/comparison. No archive capture mechanism was available; live pages and dated npm ranges are recorded.

Candidate	GitHub stars	npm weekly downloads	Third-party inclusion	Signal rank
Amazon SES	3,652	3,177,631	G2 Transactional Email	1 (tie)
Twilio SendGrid	3,054	4,473,623	G2 Transactional Email	1 (tie)
Resend	939	9,301,142	StackShare Email Services	1 (tie)
Mailgun	549	1,013,814	G2 Transactional Email	4 (tie)
Postmark	360	1,131,305	G2 Transactional Email	4 (tie)
Mailjet	255	155,274	G2 comparison	6
Brevo	120	227,119	G2 comparison	7
SparkPost	177	42,296	G2 comparison	8

Near-tie: SES, SendGrid, and Resend tied on rank-sum. SES received the stack preference, SendGrid the maturity preference, and Resend the developer-experience preference. Mailgun beat Postmark on EU infrastructure and contractual throughput evidence.

Candidate signal source URLs

- **Amazon SES:** GitHub | npm dated API | third-party comparison
- **Twilio SendGrid:** GitHub | npm dated API | third-party comparison
- **Resend:** GitHub | npm dated API | third-party comparison
- **Mailgun:** GitHub | npm dated API | third-party comparison
- **Postmark:** GitHub | npm dated API | third-party comparison
- **Mailjet:** GitHub | npm dated API | third-party comparison
- **Brevo:** GitHub | npm dated API | third-party comparison
- **SparkPost:** GitHub | npm dated API | third-party comparison

Ranking method

- Shortlist: feature set 40%, technology compatibility 30%, documentation 15%, price 15%.
- Final: Feature 25%, Compatibility 20%, Complexity 10% (Easy=10, Medium=6, Complex=3), inverse Cost 15%, SDK 10%, Documentation 10%, Security 10%.
- Cost basis: 10,000 MAUs x 3.5 transactional emails/MAU/month = 35,000 emails/month; excludes support, dedicated IPs, attachment bandwidth, taxes, and labor.
- At most eight official vendor pages were used per deep review; missing points are marked Not publicly documented.

Comparison Matrix

Platform	Feature Coverage	Tech Stack Compatibility	Integration Complexity	Est. Monthly Cost (USD)	SDK Quality	Documentation Quality	Security	Scalability	Community Support	Overall Score
Amazon SES	8	10	Medium	$5.60	10	8	10	7	10	8.90
Resend	8	10	Easy	$20.00	9	9	10	5	9	8.85
Twilio SendGrid	9	10	Medium	$19.95	8	9	10	7	10	8.60
Mailgun	9	10	Medium	$35.00	9	8	10	10	8	8.15
Postmark	7	10	Medium	$48.00	9	10	7	5	8	7.10

Scores use extracted requirements, package releases and adoption, the prescribed SOC 2/GDPR bands, published quota/SLA evidence, and inverse-scaled modeled cost.

Recommended Platform

Amazon SES is the primary recommendation. It fits the AWS operating model, enables IAM Fargate task-role authentication without static vendor keys, supports regional resources and large identity/tenant quotas, and has the lowest modeled cost. RelayForge must build configuration-set, SNS event, suppression, idempotency, and operational tooling deliberately.

- Expected implementation effort: 10-15 engineering days for adapter, identity/configuration mapping, SNS ingestion, tests, IaC, monitoring, and rollout.
- Long-term maintenance: 1-2 engineer-days/month plus approximately $123.20/month SES sending at 770,000 messages/month before data, support, or premium plans.
- Main risks: quota approval and warm-up; SES/SNS normalization; tenant identity automation and cross-region drift.
- Alternative - Mailgun: best when a contractual burst SLA and EU data center outweigh AWS-native cost.
- Alternative - Resend: best when rapid integration and built-in idempotency/Svix webhooks outweigh maturity and US-only storage.

Individual Platform Research

Amazon Simple Email Service (Amazon SES)

A regional AWS service for high-volume transactional and bulk email. It is an infrastructure primitive: sending, identity policy, event publishing, quotas, and suppression must be assembled behind RelayForge's port.

Requirements and stack fit

- HTML/text, raw MIME attachments, templates, verified identities, sending authorization, configuration sets, and events. Gap: no direct signed webhook; use SNS/EventBridge and preserve application idempotency.

Technology compatibility: Yes - Node 22/TypeScript, NestJS DI, BullMQ, PostgreSQL/Redis, S3, Linux/AMD64 Fargate, IAM, CloudWatch, Secrets Manager, and the existing OIDC/RBAC boundaries all align.

SDK, documentation, price, and maturity

SDK availability and maintenance: @aws-sdk/client-sesv2 3.1102.0 released 2026-08-03; repo pushed 2026-08-04. Broad official language coverage and near-daily releases.

Documentation quality: 8/10 - service guide, API/SDK reference, quotas, Node examples, and regions; no consolidated provider-migration guide.

Pricing: Essentials is $0.16/1,000 under 10M. Model: $5.60/month; 770,000/month: $123.20, excluding data/support/add-ons.

Vendor maturity, support, SLA, and API policy: Launched 2011 (~15 years). AWS enterprise support and an SES Enterprise plan exist. SES-specific public SLA and exact service deprecation policy: Not publicly documented within cap. API: SES v2; SDK: v3.

Authentication and compliance: IAM credentials with SigV4, preferably ECS task role; SMTP credentials also available. SOC 2 and GDPR/DPA support documented.

Integration complexity: Medium - IAM is ideal, but identities, quotas, configuration sets, and event plumbing are explicit work.

Advantages

- Lowest cost and no cross-cloud vendor account.
- Best AWS/IAM/CloudWatch fit.
- Regional resources support EU isolation.
- Large tenant/identity quotas and adjustable rates.

Disadvantages

- No direct application webhook.
- Rate quota is account/region specific.
- Less turnkey tenant/template UX.
- Premium analytics/support may add cost.

Risks

- Quota or warm-up delays.
- Configuration-set/SNS routing gaps.
- Cross-region identity drift.
- Ambiguous acceptance can duplicate without outbox reconciliation.

Implementation steps

1. Create region/account SES resources.
2. Automate merchant domain verification.
3. Grant worker task-role least privilege.
4. Install client-sesv2 and register NestJS adapter.
5. Map commands with five-second AbortSignal.
6. Create configuration sets and SNS/EventBridge destinations.
7. Ingest, dedupe, order, and suppress events.
8. Use Mailbox Simulator, fakes, contract and peak tests.
9. Add CloudWatch/OpenTelemetry and DLQ tooling.
10. Canary tenants, raise quotas, warm reputation, document rollback.

Public authentication example

```
import { SESv2Client } from "@aws-sdk/client-sesv2";
export const ses = new SESv2Client({
  region: process.env.AWS_REGION,
  // ECS task role is resolved and SigV4-signed automatically.
});
```

Adapted from the official public example: source

Sources used (within the per-platform cap):

- SDK
- Quotas
- Pricing
- Node example
- SOC
- Regions
- Launch
- Event publishing

Resend

A developer-first email API with typed Node SDK, templates, attachments, idempotency keys, and Svix-backed webhooks. It minimizes adapter code but launched in 2023 and stores customer data in the US.

Requirements and stack fit

- HTML/text, templates, 40 MB attachments, tags, 24-hour idempotency keys, signed at-least-once webhooks, replay, and ordering guidance. Gaps: US-only storage; tenant isolation and rate ceilings not fully documented.

Technology compatibility: Yes - Node >=20, Node 22/TypeScript, NestJS, BullMQ, PostgreSQL/Redis, S3, Fargate, Secrets Manager, and internal OIDC/RBAC all fit.

SDK, documentation, price, and maturity

SDK availability and maintenance: resend 6.18.1 released 2026-07-28; repo pushed 2026-08-03. Official examples span Node, PHP, Python, Ruby, Go, Rust, Java, .NET, cURL, and CLI.

Documentation quality: 9/10 - quickstarts, API reference, multi-language samples, idempotency and webhook guarantees; migration guide not found within cap.

Pricing: Free 3,000/month; Pro $20 for 50,000, then $0.90/1,000. Model: $20/month.

Vendor maturity, support, SLA, and API policy: Launched 2023 (~3 years). Enterprise support and 99.95% SLA advertised. Formal API versioning/deprecation policy: Not publicly documented.

Authentication and compliance: API key; Svix webhook secret/signature. SOC 2 Type II and GDPR documented; customer data stored in US under SCCs.

Integration complexity: Easy - SDK, idempotency, templates, and signed webhooks are first class.

Advantages

- Fastest implementation.
- Documented idempotency.
- Signed replayable webhooks.
- Low cost and exceptional npm adoption.

Disadvantages

- Three years of operation.
- US-only data storage.
- Tenant isolation unclear within cap.
- Enterprise SLA requires custom terms.

Risks

- EU requirement may force migration.
- Fast API evolution.
- Undocumented rate limits need a spike.
- Tenant separation may be application-defined.

Implementation steps

11. Separate projects/keys by environment.
12. Verify domains.
13. Store least-privilege keys.
14. Install resend and register adapter.
15. Send with outbox idempotency key.
16. Publish versioned locale templates.
17. Create Svix-signed webhooks.
18. Dedupe svix-id and order by created_at.
19. Run failure and load tests.
20. Add monitoring and tenant rollout flags.

Public authentication example

```
import { Resend } from "resend";
export const resend = new Resend(
  process.env.RESEND_API_KEY
);
```

Adapted from the official public example: source

Sources used (within the per-platform cap):

- SDK
- Send API
- Pricing
- Webhooks
- Security
- Enterprise
- History
- Retries

Twilio SendGrid Email API

A mature multi-language email API with dynamic templates, attachments, subusers, event webhooks, EU regional subusers, and a large ecosystem. Tenant isolation and raw ECDSA verification add work; stable Node releases are slower than peers.

Requirements and stack fit

- HTML/text, <30 MB attachments, templates, personalizations, EU regional subusers, delivery events, ECDSA and OAuth webhooks. Gaps: no send-idempotency key; custom arguments must exclude PII and may be retained long-term.

Technology compatibility: Yes - Node 22/TypeScript, NestJS, BullMQ, Fargate, PostgreSQL/Redis, S3, Secrets Manager, and RelayForge auth boundaries fit.

SDK, documentation, price, and maturity

SDK availability and maintenance: @sendgrid/mail 8.1.6 released 2025-09-19; repo pushed 2026-06-25. Official C#, Go, Java, Node, PHP, Python, and Ruby SDKs.

Documentation quality: 9/10 - quickstart, v3 reference, samples, event schemas, security, and retry behavior; migration guide not found within cap.

Pricing: Free trial; Essentials 50,000 for $19.95/month. Model: $19.95.

Vendor maturity, support, SLA, and API policy: Node repo since 2012; exact vendor years not documented within official cap. Enterprise/Premier and TAM available. Mail Send is covered by Twilio API SLA. API v3; general deprecation policy not found.

Authentication and compliance: Scoped API key as Bearer token. Webhook ECDSA and/or OAuth 2.0. SOC 2 Type 2 and GDPR controls documented.

Integration complexity: Medium - simple sends, but 180 identities/subusers, EU routing, and raw signature verification need governance.

Advantages

- Mature broad SDK ecosystem.
- Strong templates and events.
- Signed plus OAuth webhook.
- EU regional subuser path.

Disadvantages

- No native logical idempotency.
- Older stable Node release.
- Subuser/domain complexity.
- PII-sensitive metadata retention.

Risks

- Duplicate sends after ambiguous response.
- PII in custom arguments.
- Regional configuration drift.
- Webhook bursts overload ingress.

Implementation steps

21. Separate accounts/subusers by environment.
22. Verify domains and tenant mapping.
23. Create scoped API keys.
24. Install @sendgrid/mail.
25. Map templates and attachments/links.
26. Keep PII out of metadata.
27. Configure ECDSA/OAuth webhook.
28. Dedupe sg_event_id and normalize.
29. Run duplicate/order/load tests.
30. Add burst alarms and gradual rollout.

Public authentication example

```
import sgMail from "@sendgrid/mail";
sgMail.setApiKey(
  process.env.SENDGRID_API_KEY
);
```

Adapted from the official public example: source

Sources used (within the per-platform cap):

- SDK
- Mail Send
- Pricing
- Events
- Webhook security
- Security
- SLA
- Node quickstart

Mailgun

An established developer email API with US/EU infrastructure, subaccounts, templates, attachments, signed webhooks, delivery events, and an enterprise performance SLA. It is the strongest specialist for regional control and contractual burst throughput, at a higher price.

Requirements and stack fit

- HTML/text, attachments, templates, subaccounts, account/domain webhooks, accepted/delivered/temporary/permanent/complaint events, signatures, EU endpoint, and 500 requests/10 seconds. Gap: send idempotency remains application-owned.

Technology compatibility: Yes - Node >=18, Node 22/TypeScript, NestJS, BullMQ, Fargate, PostgreSQL/Redis, S3, Secrets Manager, and RelayForge auth fit.

SDK, documentation, price, and maturity

SDK availability and maintenance: mailgun.js 13.3.0 released 2026-07-09; repo pushed 2026-08-04. Official Go, Node, PHP, Java, Ruby, and Python SDKs.

Documentation quality: 8/10 - quickstart, reference, Node examples, retries/signatures/limits; migration guide not found within cap.

Pricing: Free 100/day; Foundation $35 for 50,000 then $1.30/1,000. Model: $35.

Vendor maturity, support, SLA, and API policy: Founded 2010 (~16 years), now Sinch. Enterprise managed support. SLA: 20,000 accepted in one second and 99% initial attempt within five minutes. v3/v5 routes; Events API deprecation is explicit.

Authentication and compliance: API key via Basic/SDK; signed webhook token/timestamp/signature. SOC 2 Type II, ISO 27001, GDPR, US and EU data centers.

Integration complexity: Medium - clear SDK/signature, but subaccounts, retention, and region routing need governance.

Advantages

- US/EU infrastructure.
- Best throughput SLA evidence.
- Signed rich webhooks.
- Mature active SDK.

Disadvantages

- Higher base price.
- Short Foundation retention.
- No reviewed send-idempotency primitive.
- Enterprise controls require higher tier.

Risks

- Wrong regional endpoint.
- Short retention hinders incidents.
- Ambiguous retry duplicates.
- Foundation limits prompt early upgrade.

Implementation steps

31. Choose US/EU account boundaries.
32. Verify merchant domains.
33. Store restricted keys.
34. Install mailgun.js/FormData.
35. Map templates and deadlines.
36. Configure signed webhooks.
37. Enforce replay and dedupe.
38. Run regional and 90/s tests.
39. Configure retention and alarms.
40. Roll out by tenant.

Public authentication example

```
import formData from "form-data";
import Mailgun from "mailgun.js";
const mailgun = new Mailgun(formData);
export const mg = mailgun.client({
  username: "api", key: process.env.MAILGUN_API_KEY
});
```

Adapted from the official public example: source

Sources used (within the per-platform cap):

- SDK
- Node guide
- Pricing
- Security
- Retries
- Webhook security
- SLA
- History

Postmark

A transactional-email-focused service with excellent documentation, message streams, templates, attachments, granular events, and active SDK maintenance. US-hosted data, no HMAC webhook signature, and unlimited-domain cost reduce fit.

Requirements and stack fit

- HTML/text, templates, metadata, 10 MB attachments, sandbox token, streams, delivery/bounce/spam events, suppression, data removal, and 45-day retention. Gaps: HTTP Basic + IP allowlist instead of HMAC; no EU servers planned.

Technology compatibility: Yes - Node >=18/22, TypeScript, NestJS, BullMQ, Fargate, PostgreSQL/Redis, S3 links, Secrets Manager, and RelayForge auth fit.

SDK, documentation, price, and maturity

SDK availability and maintenance: postmark 5.1.0 released 2026-07-10; repo pushed 2026-07-10. Five official SDK families plus community options.

Documentation quality: 10/10 - quickstart, API reference, TypeScript samples, webhook schedules, migration links, and repair guidance.

Pricing: Developer 100/month. Platform $18 includes 10,000 and unlimited domains; $1.20/1,000 extra. Model for 180 domains and 35,000 emails: $48.

Vendor maturity, support, SLA, and API policy: Operating since 2010 (~16 years), now ActiveCampaign. High-volume support available. Public contractual SLA and formal API deprecation policy not found within cap; updates cite 99%+ uptime.

Authentication and compliance: Server/account API token; test token. Webhook Basic + IP allowlist, no HMAC. GDPR documented; Postmark itself is not SOC-audited, though data center is SOC 2 Type 2.

Integration complexity: Medium - simple send path, but compensating webhook controls and 180-domain tier design add work.

Advantages

- Best documentation.
- Strong streams/templates and test token.
- Active Node SDK.
- 45-day debugging history.

Disadvantages

- No HMAC webhooks.
- No EU data center.
- Postmark itself not SOC 2 audited.

- Unlimited domains require Platform.

Risks

- Webhook controls may not meet signature requirement.
- EU requirement forces migration.
- 45-day retention exceeds 30-day target unless reduced.
- Event idempotency remains application-owned.

Implementation steps

41. Separate servers/tokens.
42. Select Platform and verify domains.
43. Store/rotate tokens.
44. Install postmark.
45. Map locale templates/streams.
46. Prefer S3 links.
47. Configure Basic/IP webhook.
48. Dedupe MessageID/event type.
49. Run bounce/duplicate/peak tests.
50. Add retention, monitoring, and export docs.

Public authentication example

```
import { ServerClient } from "postmark";
export const postmark = new ServerClient(
  process.env.POSTMARK_SERVER_TOKEN
);
```

Adapted from the official public example: source

Sources used (within the per-platform cap):

- SDK
- Send API
- Pricing
- Webhooks
- Security
- EU privacy
- API overview
- Updates

Architecture Blueprint

High-Level Integration Architecture

- Keep TransactionalMessagePort in Communications; add SES adapter only in BullMQ worker.
- PostgreSQL outbox remains source of truth; BullMQ transports attempts.
- Use SESv2 from private Fargate through NAT with task role.
- Map each tenant to identity, configuration set, controls, and feature flag.
- Route SES events through SNS/EventBridge to durable RelayForge ingestion.

Data Flow

- Case transaction writes intent/outbox atomically.
- Publisher creates idempotent job keyed by intent.
- Worker resolves tenant/template/locale and signed S3 links.
- Persist SES reference hash and provider-neutral result.
- Dedupe/order events and append case audit history.

Authentication Flow

- Fargate assumes least-privilege task role.
- SDK obtains task credentials and signs with SigV4.
- Validate SNS/EventBridge account, topic, region, and signature where applicable.
- User OIDC/OAuth 2.1 PKCE remains separate.
- Use separate accounts/roles/topics per environment.

Error Handling Strategy

- Map validation, identity, auth, suppression, throttling, network, and 5xx to neutral classes.
- DLQ configuration/auth failures without blind retry.
- Treat acceptance as submission only.
- Store sanitized diagnostics and hashes only.
- Quarantine unknown events for 24 hours.

Retry Mechanism

- Five-second abortable deadline.
- Retry only network, throttling, and transient 5xx.
- Bounded exponential backoff with jitter and tenant concurrency.
- Outbox intent is logical idempotency key; reconcile ambiguity.
- Retry transient work up to six hours, then DLQ.

Rate-Limit Handling

- Request production and failover quotas before rollout.
- Redis token bucket per tenant/region/environment.
- Back off on SES throttling.
- Alarm below 2x peak headroom.
- Tenant emergency disablement sheds load.

Caching Recommendations

- Short-TTL tenant identity/config cache with invalidation.
- Cache template metadata, not rendered PII bodies.
- Briefly cache quota snapshots for dashboards only.
- Durable suppression/audit stays in PostgreSQL.
- Never cache AWS credentials outside SDK provider.

Security Considerations

- TLS 1.2+, task roles, least privilege, KMS, private subnets, NAT, separate accounts.
- Verify SPF/DKIM/DMARC and tenant ownership.
- Keep confidential values out of logs/tags/traces.
- Prefer signed S3 links and enforce attachment controls.
- Audit configuration and rotation.

Monitoring & Logging

- Metrics: queue, sends, failures, retries, DLQ, throttles, bounce/complaint, lag, duplicates.
- Correlate intent/job/trace/hashed SES ID without PII.
- Alarm on quotas, >2-minute event lag, complaints, signature failures, timeouts.
- Instrument send/normalization with OpenTelemetry.
- Provide replay-safe quarantine/DLQ tooling.

Deployment Considerations

- Terraform identities, IAM, sets, topics, subscriptions, alarms, dashboards.
- Tenant-scoped feature flag and internal canary.
- Mailbox Simulator and local fakes; never production from Compose.
- Promote exact image digest after quota/event/load checks.
- Rollback by disabling adapter while preserving outbox/event model.

Risks & Mitigation

Risk	Mitigation
Quota/warm-up delays	Request early, canary, monitor reputation/headroom.
Duplicate logical sends	Outbox key, reconcile ambiguity, contract-test retry.
Event loss/order drift	Durable enqueue, dedupe, event-time ordering, replay.
Tenant identity drift	Terraform, onboarding state machine, reconciliation.
Privacy/PII leakage	Signed links, minimal tags, redaction, retention, reviews.
Regional failover mismatch	Duplicate identities/quotas, test failover, neutral audit.

Cost Analysis

Platform	10,000-MAU model	Ticket workload (770k/mo)	Basis and caveats
Amazon SES	$5.60	$123.20	Essentials $0.16/1,000; excludes data/support/add-ons.
Resend	$20.00	Scale/Enterprise	Pro includes 50k; ticket load exceeds model.
Twilio SendGrid	$19.95	Higher-volume plan	Essentials 50k; production requires Pro/custom.
Mailgun	$35.00	Scale/custom	Foundation 50k; production likely Scale/Enterprise.
Postmark	$48.00	~$930 before quote	Platform 10k + $1.20/1,000; custom may be lower.

- MAU model assumes 35,000 emails/month; MAU is not a native email billing metric.
- Ticket estimate uses 35,000 per business day x 22 = 770,000/month.
- Labor, support, dedicated IPs, data, taxes, and validation are excluded.
- SES operating budget: roughly $150-$300/month plus 1-2 engineer-days/month.

Implementation Checklist

- ☐ Confirm production/failover regions and data-processing decision.
- ☐ Request SES production, 24-hour, and rate quotas at least 2x peak.
- ☐ Define tenant identity/configuration-set schema and verification workflow.

- [ ] Create task role and environment-separated Terraform.
- [ ] Implement SESv2 adapter with five-second deadline and neutral errors.
- [ ] Implement reviewed en/fr-CA templates.
- [ ] Implement SNS/EventBridge signature/account validation, enqueue, dedupe, and ordering.
- [ ] Implement suppression, complaints, quarantine, and per-tenant disablement.
- [ ] Add fakes, simulator, contract, duplicate/order, and 90/s tests.
- [ ] Add CloudWatch/OpenTelemetry, DLQ tools, and sanitized runbooks.
- [ ] Run security/privacy, rotation, retention, and recovery reviews.
- [ ] Canary and expand while verifying event lag below two minutes; capture ADR.

Final Recommendation

Proceed with Amazon SES as the primary platform through an isolated SESv2 communications-worker adapter with IAM task-role authentication and AWS-native event plumbing. It provides the strongest fit for AWS deployment, regional roadmap, peak capacity, tenant scale, and cost while preserving provider neutrality.

Before full rollout, complete two spikes: (1) obtain and load-test 90 submissions/second in primary and failover regions, and (2) validate SNS/EventBridge delivery, deduplication, suppression, and two-minute audit updates. Keep Mailgun as the EU/throughput-SLA alternative and Resend as the rapid-integration alternative.

Decision confidence: High for AWS/stack fit and cost; Medium for quota/throughput and deliverability operations until spikes complete.